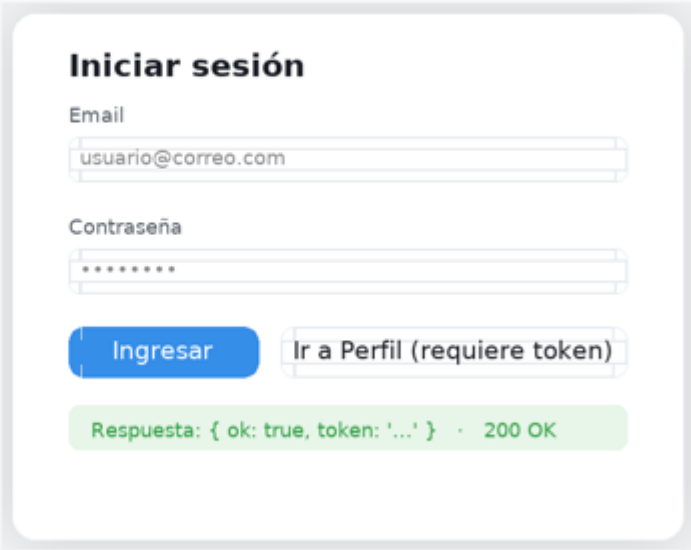


## 1) Login — POST /auth/login (emite JWT)

Muestra un formulario con **email** y **contraseña**. Al enviar, el backend valida credenciales y responde `{ ok, token }`. El token se guarda en memoria/localStorage. Códigos esperados: **200 OK** (éxito), **400/401** (faltan datos o credenciales inválidas).

### M8 · AE4 — Login (JWT)

POST /auth/login → devuelve token, guardar token en memoria/localStorage



The mockup shows a login form with the following elements:

- Title:** Iniciar sesión
- Email:** A text input field containing the placeholder text "usuario@correo.com".
- Contraseña:** A password input field with masked characters "\*\*\*\*\*".
- Buttons:** A blue "Ingresar" button and a light gray "Ir a Perfil (requiere token)" button.
- Response:** A green box at the bottom displays the response: "Respuesta: { ok: true, token: '...' } · 200 OK".

## 2) Perfil (ruta protegida) — GET /api/perfil

Pantalla visible **solo con token válido** enviado en el header `Authorization: Bearer <token>`. Muestra datos básicos del usuario (email/rol) y un botón **Cerrar sesión**. Código esperado: **200 OK** con el payload del usuario.

### M8 · AE4 — Perfil (ruta protegida)

GET /api/perfil con Authorization: Bearer <token válido>

#### Perfil de usuario

Token válido ✓ · Authorization: Bearer eyJhbGciOi...

Email: demo@mail.com

Rol: user

Expira: en 15 minutos

Cerrar sesión

Respuestas esperadas: 200 OK con datos del usuario

### 3) Acceso denegado — 401

Estado cuando **falta el token** o es **inválido/expirado**. Se indica el header requerido y se sugiere volver a **Login**. Código esperado: **401 Unauthorized** (o **403** si decides diferenciar permisos).

